

Capstone Task 4: Analisi SQL Injection con Wireshark

Data: 21/02/2026

Analista: Daniele di Martino - Bkm4ge

Introduzione

Nella task odierna viene richiesto di analizzare un incidente con wireshark.

Gli attacchi di SQL injection consentono agli hacker malintenzionati di digitare istruzioni SQL in un sito web e ricevere una risposta dal database. Ciò permette agli aggressori di manomettere i dati correnti nel database, falsificare identità e compiere varie azioni dannose.

Step 1: Analisi del file

Una volta aperto Wireshark, il primo passo è notare in quanto tempo siano state fatte le richieste, nel caso di questo attacco la durata è stata di circa 441 secondi (8 minuti circa), vengono identificati gli IP Attaccante e Target:

IP Attaccante: 10.0.2.4

IP Target: 10.0.2.15

Viene fatta una richiesta SYN, quindi con protocollo TCP per comunicare col target

Porta inizio comunicazione: 35614

Porta trasmissione: 80

Richiesta fatta su canale in chiaro attraverso protocollo HTTP

fase 1 dell'attacco: Attraverso l'inserimento della stringa 1=1 (tipica delle SQL Injection), il find restituirà il tentativo nell'header:



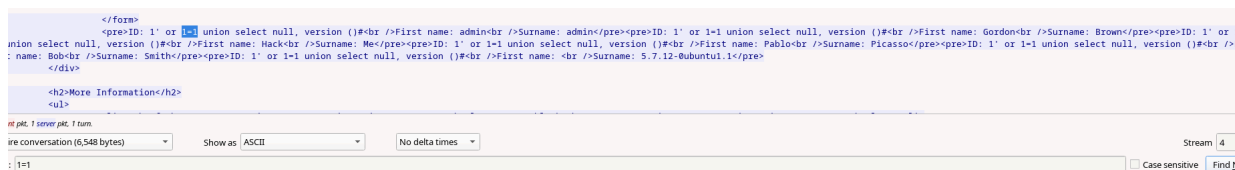
Prova inconfutabile del tentativo di manomissione.

fase 2 dell'attacco:

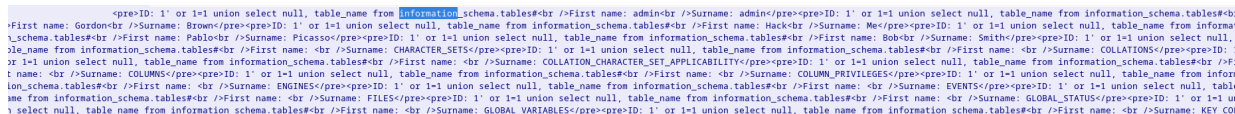


Il nome del database è dvwa e l'utente del database è root@localhost. Vengono visualizzati anche più account utente.

fase: 3 versione DVWA: **v1.10 *Development***



fase 4: restituzione con le tabelle degli utenti



Attraverso Crackstation e l'estrazione del hash **5f4dcc3b5aa765d61d8327deb882cf99** viene restituito **password**

Conclusioni

1. Valutazione del Rischio Sistemico: L'Impiego di SQL

Il rischio fondamentale nell'utilizzo di database relazionali basati su SQL all'interno di piattaforme web risiede nella potenziale sovrapposizione tra il **piano dei dati** (l'input fornito dall'utente) e il **piano di controllo** (la sintassi eseguibile della query).

Quando un'applicazione web costruisce istruzioni SQL concatenando dinamicamente stringhe di testo senza un'adeguata validazione strutturale, genera un'anomalia. Questa vulnerabilità architetturale permette a un input esterno di alterare la logica semantica dell'interrogazione originale. La gravità di tale anomalia (che spazia dall'esfiltrazione dei dati all'esecuzione di codice in remoto) è determinata dai privilegi concessi all'account del database e dall'obiettivo dell'attaccante.

2. Metodologie di Prevenzione: Analisi delle Contromisure

Per neutralizzare le relazioni causali che portano a un'iniezione SQL, è necessario implementare un paradigma di *defense-in-depth*. Tra le pratiche operative individuate, le seguenti due rappresentano gli strati difensivi più efficaci a livello strutturale:

- **Utilizzo di Parametri (Prepared Statements / SQL Dinamico Parametrizzato):**
 - **Logica Operativa:** Questo approccio forza una separazione rigorosa tra il codice eseguibile e i dati. Il motore del database compila la struttura della query SQL prima ancora di ricevere l'input dell'utente.
 - **Impatto Sistemico:** Qualsiasi input inserito nei parametri viene trattato dal motore del database esclusivamente come un valore letterale (dato) e mai come un comando eseguibile, disinnescando alla radice la possibilità di alterare la query.
- **Implementazione di Web Application Firewall (WAF) e Filtraggio dell'Input:**
 - **Logica Operativa:** Il WAF opera come filtro perimetrale analizzando il traffico HTTP in entrata alla ricerca di pattern comportamentali anomali o firme note di iniezione. Il filtraggio dell'input (preferibilmente tramite *whitelist*) agisce a livello applicativo, garantendo che i dati accettati rispettino formati, lunghezze e tipologie rigorosamente predefiniti.
 - **Impatto Sistemico:** Interrompe la catena di attacco respingendo i payload malevoli a livello di rete o di applicazione, prima che questi raggiungano lo strato logico del database.

In ottica di contenimento del danno (*hardening*), la **disabilitazione di funzionalità non necessarie del database** (es. procedure archiviate di sistema non utilizzate o comandi di esecuzione shell esterna) serve a limitare i movimenti laterali e le capacità operative dell'aggressore qualora la vulnerabilità primaria venisse compromessa.

